ChatGPT

Customizing Synonym Behavior in Azure AI Search

Introduction

Modern search applications must interpret user queries even when the vocabulary used in the query differs from the content in the index. Azure AI Search provides two mechanisms for handling synonyms: **synonym maps**, which expand a query at run-time, and **custom analyzers** with a `SynonymTokenFilter`, which embed synonyms into the index at indexing time. Understanding the differences between these mechanisms and how to implement them is critical for building robust search experiences. This report summarizes available customization options, provides practical examples, and presents a proof-of-concept demonstrating how to define synonym rules, attach them to fields and analyzers, and test their effect on search results.

Background: synonyms in Azure AI Search

The lexical analysis pipeline in Azure AI Search converts text into tokens that can be indexed or compared against search queries. Synonym support helps align search queries with the vocabulary of your content. There are two main strategies:

1. **Query-time synonym expansion via synonym maps.** A synonym map is a top-level resource containing rules in Solr format. When a query term matches a synonym rule, the query is rewritten to include equivalent or mapped terms before it is executed [1]. Equivalency rules treat all terms in a rule as equal substitutes, while explicit mapping rules rewrite specific phrases to a canonical term [2]. A field definition in the index can reference a synonym map through its `synonymMaps` property, allowing the map to be applied without rebuilding the index [3].

2. **Index-time synonym expansion via custom analyzers.** A custom analyzer lets you combine a tokenizer with a sequence of token and character filters. The `SynonymTokenFilter` matches single or multi-word synonyms during analysis [4]. Unlike synonym maps, synonyms defined in a token filter are applied when documents are indexed (or optionally at query time), expanding tokens in the index itself. Multi-word synonyms and other complex tokenization scenarios can be handled at this stage.

Synonym maps (query-time expansion)

- **Resource definition and format.** A synonym map has three parts: a name, a format and a set of rules. The only supported format is `solr` [2]. Rules are written using Solr's synonym grammar and can be of two kinds:
- **Equivalency rules** (comma-separated list) treat terms as equal substitutes. For example, the rule `dog, puppy, canine` causes a query on **"canine"** to expand to **"canine OR dog OR puppy"** [5].
- **Explicit mapping rules** use `=>` to rewrite a phrase to another form. The rule `Washington, Wash., WA => WA` rewrites "Washington" and "Wash." to "WA" at query time [6].

- **Limits.** Synonym maps in the Free tier can contain up to 5,000 rules; other tiers support 20,000 rules per map, and each rule can have up to 20 expansions [7]. Creating, updating or deleting a synonym map is a whole-document operation—you cannot modify individual rules [8].
- **Assigning a map to fields.** Synonym maps apply only to searchable string fields. A field must be of type `Edm.String` or `Collection(Edm.String)` and set `searchable` to `true`. Only one synonym map can be assigned per field [9]. Once a synonym map is attached to a field, queries over that field automatically expand or rewrite terms according to the map [10].
- **Query behavior.** Synonyms are expanded only for free-form text queries; they do not apply to filters, facets, autocomplete or suggestions [11]. Internally the search engine rewrites the query using the OR operator, so synonyms contribute equally to scoring and highlighting [12]. Wildcard, fuzzy or regex searches are not expanded automatically but can be combined manually using OR syntax [13].

Custom analyzers and the `SynonymTokenFilter` (index-time expansion)

Custom analyzers give developers fine-grained control over tokenization. An analyzer combines a tokenizer, optional character filters and token filters. The `SynonymTokenFilter` is a Lucene-based filter that matches single or multi-word synonyms in the token stream [4]. It accepts a list of synonym rules in the same two formats as synonym maps (comma-separated equivalency or `=>` explicit mappings). Important properties include:

- `synonyms` **(required)** – list of synonym rules. For example: `incredible, unbelievable, fabulous => amazing` rewrites the terms on the left to "amazing"; `incredible, unbelievable, fabulous, amazing` defines a set of equivalent words [14].
- `expand` **(default** `True` **)** – controls whether all words in the list map to one another. When `expand` is `True`, `incredible, unbelievable, fabulous, amazing` expands to `incredible OR unbelievable OR fabulous OR amazing`; when `False`, the rule is interpreted as mapping all synonyms to the first term [15].
- `ignoreCase` **(default** `False` **)** – whether to case-fold input for matching [14].

Custom analyzers are defined in the index schema. The following example from a Stack Overflow answer demonstrates how to use a custom analyzer with a `SynonymTokenFilter` to support multi-word synonyms. When indexing a document containing "wifi," the analyzer emits both "wifi" and "wi fi," allowing queries for either term to match the document. The analyzer definition in JSON looks like this [16]:

```
"analyzers": [
  {
    "@odata.type": "#Microsoft.Azure.Search.CustomAnalyzer",
    "name": "synonym-analyzer",
    "tokenizer": "microsoft_language_tokenizer",
    "tokenFilters": [ "synonym-filter" ]
  }
],
"tokenFilters": [
  {
    "@odata.type": "#Microsoft.Azure.Search.SynonymTokenFilter",
    "name": "synonym-filter",
    "synonyms": [ "wifi,wi-fi,internet,wi fi" ],
```

```
    "expand": true
  }
]
```

This analyzer can be applied at indexing time, query time or both. Because the synonyms are stored in the index, multi-term expansions work correctly. One limitation is that changing the synonym list requires recreating the index, whereas synonym maps can be edited independently [17].

Comparison of query-time vs index-time synonym expansion

Aspect	Synonym map (query-time)	Custom analyzer with `SynonymTokenFilter` (index-time)
Where synonyms are applied	Query parser rewrites the query before execution [12].	Analyzer expands tokens during indexing (and optionally query processing) [4].
Update flexibility	Synonym maps are independent resources; you can add or remove rules without reindexing [10].	Synonyms are embedded in the index; changing them requires rebuilding the index [17].
Supported term patterns	Single words and phrases; multi-word synonyms rely on phrase queries and proper quoting [5].	Handles single and multi-word synonyms in the token stream [4].
Control over analysis	Uses the index's assigned analyzer; synonyms are subject to the same tokenization and filters [18].	Complete control over tokenizer and filters, enabling custom tokenization, case folding, stemming, etc. [14].
Use cases	Quickly adjust synonyms for search features like synonyms for jargon, slang or brand names. Suitable for equivalency or explicit rewriting.	Embedding synonyms into the index for multi-word or complex expansions, phonetic search, or domain-specific normalization.

Proof-of-Concept (PoC)

The following PoC illustrates how to customize synonyms in Azure AI Search using both query-time synonym maps and index-time custom analyzers. It uses the Python `azure-search-documents` SDK (v11). Although the code cannot be executed here without access to an Azure Search service, it demonstrates the steps required.

1 – Create a synonym map

```
from azure.search.documents.indexes import SearchIndexClient
from azure.search.documents.indexes.models import SynonymMap, SimpleField,
SearchFieldDataType, SearchableField, ComplexField, SearchIndex
from azure.core.credentials import AzureKeyCredential
```

```
# Set these variables to your service information
service_endpoint = "https://<your-search-service>.search.windows.net"
api_key = "<your-admin-key>"

# Create the client
client = SearchIndexClient(endpoint=service_endpoint,
credential=AzureKeyCredential(api_key))

# Define the synonym map in Solr format
synonym_rules = """
    USA, United States, United States of America
    dog, puppy, canine
    Washington, Wash., WA => WA
""".strip()

synonym_map = SynonymMap(
    name="demo-synonyms",
    format="solr",
    synonyms=synonym_rules
)

# Create or update the synonym map
client.create_synonym_map(synonym_map)

print("Synonym map created.")
```

This snippet creates a synonym map named `demo-synonyms` with equivalency rules (USA ↔ United States ↔ United States of America and dog ↔ puppy ↔ canine) and an explicit mapping rule (Washington and Wash. are rewritten to WA). The `create_synonym_map` call creates or replaces the map [19].

2 – Create an index and assign the synonym map

```
# Define fields for the index.  Only searchable string fields can use synonyms.
fields = [
    SimpleField(name="hotelId", type=SearchFieldDataType.String, key=True),
    SearchableField(name="hotelName", type=SearchFieldDataType.String,
filterable=True, sortable=True),
    SearchableField(name="description", type=SearchFieldDataType.String,
analyzer_name="en.lucene", synonym_map_names=["demo-synonyms"]),
    ComplexField(name="address", fields=[
        SearchableField(name="streetAddress", type=SearchFieldDataType.String),
        SearchableField(name="city", type=SearchFieldDataType.String),
        SearchableField(name="stateProvince", type=SearchFieldDataType.String),
        SearchableField(name="country", type=SearchFieldDataType.String,
synonym_map_names=["demo-synonyms"]),
```

```
    ])
]

index = SearchIndex(name="hotels-index", fields=fields)
client.create_index(index)
print("Index created with synonym map assignments.")
```

This code defines a `hotels-index` with a `description` field and a nested `country` field that both reference the `demo-synonyms` map. After creation, any query that searches those fields expands or rewrites terms according to the map [20].

3 – Query the index with synonyms

```
from azure.search.documents import SearchClient

search_client = SearchClient(endpoint=service_endpoint, index_name="hotels-
index", credential=AzureKeyCredential(api_key))

# Example query for canine.  Because of the synonym rule, this finds documents
containing dog or puppy.
results = search_client.search("canine")

for result in results:
    print(result["hotelId"], result["description"])

# Example query for Washington.  Because of the explicit rule, the query is
rewritten to "WA".
results = search_client.search("Washington")
```

When the client sends the query **“canine”**, the service expands the query to **“canine OR dog OR puppy”**; when the query is **“Washington”** or **“Wash.”**, the engine rewrites it to **“WA”** before execution [21]. No changes are required to the index; the synonym map is applied at query time.

4 – Use a custom analyzer with `SynonymTokenFilter` for multi-word synonyms

Multi-word synonyms or expansions that combine multiple tokens can be handled more effectively at indexing time. The following example defines a custom analyzer to index the phrase **“wi fi”** whenever the term **“wifi”** appears. It uses the `microsoft_language_tokenizer` tokenizer and a `SynonymTokenFilter`:

```
from azure.search.documents.indexes.models import CustomAnalyzer, TokenFilter

syn_filter = TokenFilter(
    odata_type="#Microsoft.Azure.Search.SynonymTokenFilter",
    name="wifi-synonyms",
```

```
    synonyms=["wifi,wi-fi,internet,wi fi"],
    expand=True
)

custom_analyzer = CustomAnalyzer(
    name="wifiAnalyzer",
    tokenizer="microsoft_language_tokenizer",
    token_filters=[syn_filter.name],
    char_filters=[]
)

fields = [
    SimpleField(name="docId", type=SearchFieldDataType.String, key=True),
    SearchableField(name="content", type=SearchFieldDataType.String,
analyzer_name=custom_analyzer.name)
]

index = SearchIndex(name="wifi-index", fields=fields,
analyzers=[custom_analyzer], token_filters=[syn_filter])
client.create_index(index)

print("Index with custom analyzer created.")
```

During indexing, the analyzer tokenizes the content and applies the synonym filter. For a document containing “wifi,” the tokens emitted include both “wifi” and “wi fi.” This ensures that queries for the multi-word phrase **“wi fi”** match the document, even though the original text contains “wifi.” Updating the synonym filter later would require recreating the index because the synonyms are stored in the inverted index [17].

5 – (Optional) Query with the custom analyzer at search time

If you want queries themselves to be normalized using the same analyzer, you can specify the `searchAnalyzer` property on the field. For example, setting `searchAnalyzer="wifiAnalyzer"` on the `content` field causes queries like **"wi-fi"** or **"internet"** to be analyzed with the synonym filter. Otherwise, you may rely on the index-time expansion and use a standard analyzer for queries.

Diagram: overview of synonym customization strategies

The following diagram summarizes how query-time and index-time synonym expansion work. At the top, a user query is rewritten using a synonym map before hitting the search index. At the bottom, document content is processed by a custom analyzer with a `SynonymTokenFilter` to store expanded terms in the index. The search engine retrieves results by matching the rewritten query against the index. A copy of this diagram is provided below.

Considerations and best practices

- **Choose the right strategy.** Use synonym maps for simple equivalency or explicit rewrites that need to be updated frequently without reindexing. Use custom analyzers with synonym filters for multi-word synonyms, phonetic expansions or domain-specific tokenization.
- **Limit the scope of synonyms.** Attach synonym maps only to fields where expansion is needed. Overuse of synonyms can reduce precision and degrade relevance.
- **Test and iterate.** Experiment with small synonym dictionaries in development, measure impact on scoring and query latency, and refine the rules accordingly [22].
- **Combine with vector/semantic search (optional).** Synonyms can complement semantic search by expanding queries before embedding them. However, because semantic models already handle synonyms to some extent, test whether additional synonym expansion improves or degrades results.

Conclusion

Azure AI Search offers flexible options for customizing synonyms. **Synonym maps** provide a declarative way to expand or rewrite query terms at run-time without reindexing; they support equivalency and explicit mappings defined in Solr format [23]. **Custom analyzers** with the `SynonymTokenFilter` integrate synonyms into the indexing pipeline, enabling multi-word expansions and more control over tokenization [4]. Choosing between these approaches depends on your requirements for update flexibility, complexity of synonym rules and control over analysis. The PoC and diagram in this report should serve as a starting point for implementing synonym customization in your own search applications.

---

[1] [2] [3] [5] [6] [7] [8] [9] [10] [11] [12] [13] [18] [20] [21] [22] [23] Add synonyms to expand queries for equivalent terms - Azure AI Search | Microsoft Learn
https://learn.microsoft.com/en-us/azure/search/search-synonyms

[4] [14] [15] Add custom analyzers to string fields - Azure AI Search | Microsoft Learn
https://learn.microsoft.com/en-us/azure/search/index-add-custom-analyzers

[16] [17] Is there a way to make multi term synonyms work in Azure Cognitive Search - Stack Overflow
https://stackoverflow.com/questions/74223109/is-there-a-way-to-make-multi-term-synonyms-work-in-azure-cognitive-search

[19] raw.githubusercontent.com
https://raw.githubusercontent.com/Azure/azure-sdk-for-python/main/sdk/search/azure-search-documents/samples/async_samples/sample_synonym_map_operations_async.py